# Systematic Design of RAG-based LLM Inference Systems: A Survey

REPOSITORY DRAFT, Awesome-RAG-LLM-Inference-System Repository, GitHub

Retrieval-augmented generation (RAG) has become a practical way to improve large language model (LLM) responses with external knowledge, but its deployment cost is increasingly shaped by systems concerns rather than model design alone. Modern RAG serving pipelines must coordinate query encoding, vector retrieval, reranking, prompt construction, and autoregressive decoding under tight latency, memory, and cost constraints. Recent work shows a clear shift from model-only optimization toward end-to-end co-design across accelerators, caches, schedulers, and storage systems. This survey organizes the repository literature into a unified taxonomy of RAG inference system design. We summarize the main system bottlenecks, review five major design axes, provide comparative discussion with explicit citations, and include framework figures for the end-to-end pipeline, the design taxonomy, and the memory hierarchy. The survey closes with cross-cutting trade-offs, research trends, and open challenges for building efficient, scalable, and quality-aware RAG inference systems.

Additional Key Words and Phrases: retrieval-augmented generation, large language models, inference systems, vector search, memory hierarchy, systems survey

## 1 INTRODUCTION

Retrieval-augmented generation has become a core architecture for knowledge-intensive LLM applications because it separates parametric generation from non-parametric knowledge storage. Instead of depending only on model weights, a RAG pipeline retrieves relevant external context, injects it into the prompt, and then decodes a response conditioned on both the query and the retrieved evidence. This design can improve factuality, freshness, domain adaptation, and controllability, but its practical value now depends heavily on end-to-end systems efficiency rather than retrieval quality alone.

The recent literature collected in this repository reflects a clear system-centric shift. End-to-end serving papers such as RAGO and Hermes treat RAG as a coordinated retrieval and generation pipeline rather than an isolated retrieval module [14, 25]. Cache-centric systems such as Cache-Craft, CacheBlend, MemoRAG, HyperRAG, CacheFocus, and METIS show that reuse, adaptation, and cache management increasingly determine both quality and cost [3–5, 15, 18, 19]. Scheduling-oriented work such as PipeRAG, RAGDoll, ELERAG, Patchwork, AquaPipe, and DGRAG demonstrates that overlap among retrieval, prompt construction, and decoding can be as important as speeding up any single stage [2, 9, 12, 21, 23, 24]. Meanwhile, SmartSSD, near-data, and disk-native vector-search systems show that large-scale RAG cannot rely on GPU memory alone [7, 10, 16, 17, 20, 27].

This survey provides a system-oriented synthesis of that design space. Our contributions are:

(1) a taxonomy of RAG inference system designs spanning GPU acceleration, reuse, scheduling, external memory, and application-specific optimization;
(2) a comparison of architectural trade-offs across latency, throughput, memory capacity, quality, and freshness; and
(3) a summary of research trends and open challenges for scalable, efficient, and quality-aware RAG serving.

Author's address: Repository Draft, Awesome-RAG-LLM-Inference-System Repository, Online, GitHub.

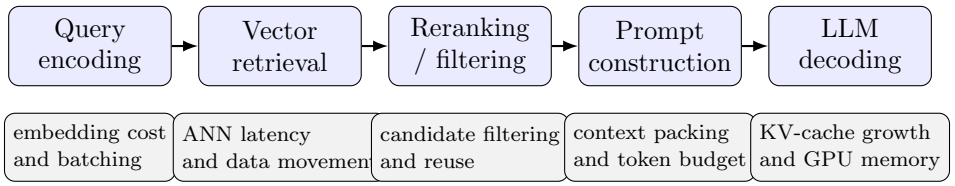


Fig. 1. A typical RAG inference pipeline. Practical serving stacks must coordinate retrieval-side latency, prompt-side orchestration, and decode-side memory pressure rather than optimizing only one stage.

## 2 BACKGROUND: RAG INFERENCE AS A SYSTEMS PIPELINE

An end-to-end RAG inference pipeline usually contains five stages: query encoding, vector retrieval, reranking or filtering, prompt construction, and LLM decoding. Although that decomposition is conceptually simple, each stage stresses a different part of the underlying system stack. Figure 1 illustrates the typical pipeline and the main bottlenecks exposed at each stage.

### 2.1 Latency, memory, and throughput bottlenecks

Vector search can dominate latency when the corpus is large or when the index does not fit in fast memory. That pressure is visible in GPU-native search designs such as CAGRA, PilotANN, and VecFlow, which attempt to improve retrieval efficiency through accelerator-aware data structures and kernels [6, 22, 29]. In end-to-end RAG systems, however, retrieval is only one part of the path. RAGO and Hermes show that CPU/GPU transfers, scheduling gaps, and prompt assembly overhead can materially affect end-to-end performance even when the underlying ANN component is fast [14, 25].

Memory is the second persistent bottleneck. RAG serving must simultaneously manage vector indexes, retrieved context, and expanding LLM KV caches. As corpora and context windows grow, accelerator memory becomes insufficient, which motivates disaggregated and hierarchical memory designs such as Chameleon, SmartSSD-based search, DiskANN, and LSM-VEC [7, 10, 17, 27]. Throughput is the third bottleneck: if retrieval, reranking, and decoding have mismatched service rates, queues accumulate and hardware remains underutilized [2, 21, 23].

### 2.2 Key evaluation metrics

A complete RAG-system evaluation should include average latency, tail latency, throughput under concurrency, answer quality or recall, memory footprint, cost efficiency, and freshness support. Recent work has also made energy efficiency and quality-aware control more visible, especially for hardware-specialized or adaptive systems [19, 20, 27]. These metrics motivate the taxonomy in Figure 2.

## 3 TAXONOMY OF RAG INFERENCE SYSTEM DESIGN

### 3.1 GPU acceleration

The first line of work adapts vector search and end-to-end RAG serving to GPU architectures. Systems such as RAGO and Hermes illustrate the end-to-end view: retrieval, data movement, prompt handling, and generation are jointly optimized around the accelerator [14, 25]. At the algorithm level, CAGRA, PilotANN, and VecFlow show how GPU-oriented graph search, partitioning, and filtered vector data management can increase parallelism and reduce

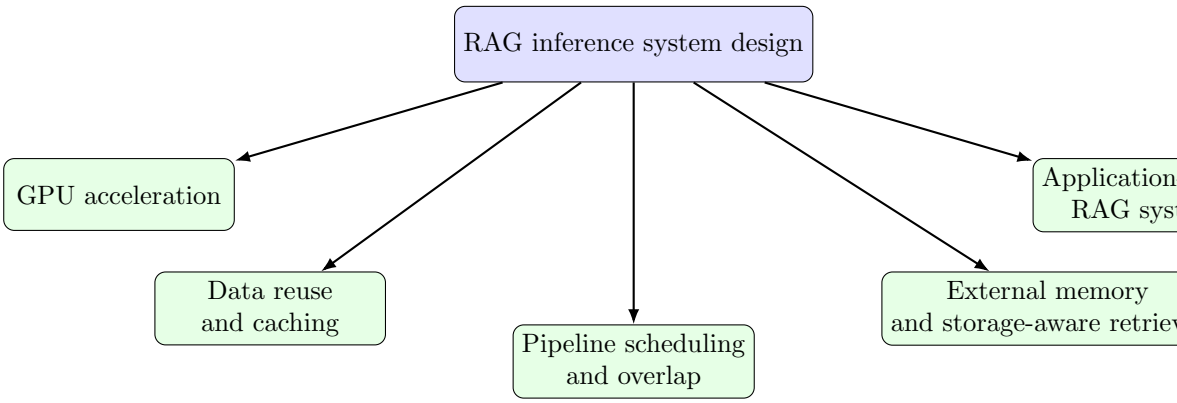


Fig. 2. Taxonomy used in this survey. The current literature clusters around accelerator-aware retrieval, reuse-centric serving, scheduling and pipelining, memory-hierarchy expansion, and workload-specific system design.

memory stalls [6, 22, 29]. Related CPU/GPU collaborative designs highlight that hybrid placement remains attractive when the retrieval corpus is large or when preliminary filtering can reduce accelerator load [8].

Three themes recur in this category. First, GPU-native data structures often outperform CPU-derived designs once search order, graph layout, and memory access patterns are reorganized for SIMT execution [6]. Second, end-to-end overlap matters: a fast retrieval kernel does not automatically yield a fast RAG service if transfers and prompt orchestration remain serialized [14, 25]. Third, accelerator capacity remains the main limitation, which naturally motivates cache reuse and external memory.

### 3.2 Data reuse and cache-centric optimization

The second design axis treats reuse as a primary systems strategy. Cache-Craft and CacheBlend focus on knowledge reuse directly within the serving stack, showing that repeated access to documents or fused cached knowledge can substantially reduce end-to-end latency [3, 4]. MemoRAG and HyperRAG extend this view by reusing memory summaries and reranker KV caches to improve quality-efficiency trade-offs in long-context or reranking-heavy settings [15, 18]. CacheFocus and METIS demonstrate that static caches are often insufficient: efficient RAG systems increasingly need dynamic repositioning and quality-aware configuration control [5, 19].

This literature spans several reuse targets: retrieved evidence, prompt-side context, reranker states, decoder KV caches, and operating configurations. The shared insight is that many workloads exhibit temporal or semantic locality. Yet reuse must be balanced against freshness and storage cost. EdgeRAG and SPFresh, for example, highlight the importance of online updates and evolving indexes when the knowledge base changes over time [11, 28]. As a result, caching is no longer a minor optimization; it is becoming a first-class architectural component of RAG serving.

### 3.3 Pipeline parallelism and scheduling

The third line of work studies how retrieval and generation interact in time. PipeRAG, RAGDoll, ELERAG, AquaPipe, Patchwork, and DGRAG all show that overlapping retrieval,

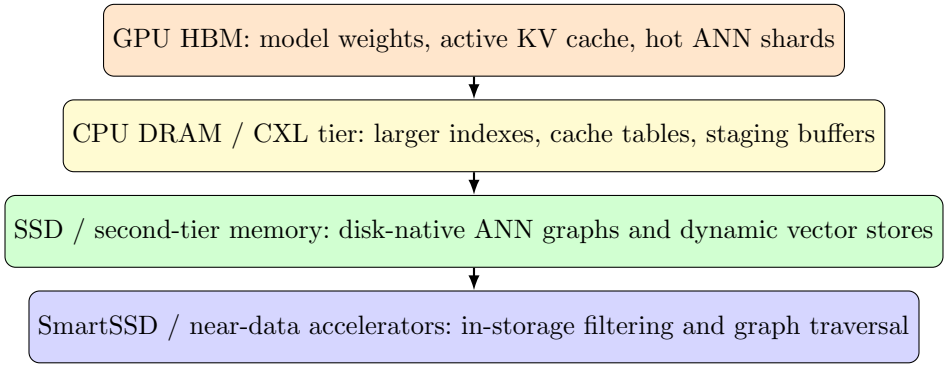


Fig. 3. Memory hierarchy for large-scale RAG inference. Current systems increasingly span accelerator memory, host memory, second-tier memory, SSD-resident indexes, and near-data processing.

prompt construction, and decoding can significantly improve user-visible performance [2, 9, 12, 21, 23, 24]. Some systems rely on adaptive pipelining, some on offloading, and some on lookahead retrieval or distributed placement, but they all emphasize orchestration rather than isolated component speed.

This category also reveals the limits of naive overlap. Retrieval difficulty varies from query to query; speculative retrieval can waste work; and heterogeneous systems are harder to balance under changing load [9, 12, 23]. Even so, the broader lesson is important: queueing behavior, overlap, and resource assignment can influence end-to-end latency as much as the underlying ANN algorithm.

### 3.4 External-memory and storage-aware acceleration

Large-scale RAG frequently exceeds the memory budget of a single accelerator or server. The fourth axis therefore studies how to push computation closer to storage or how to extend the effective memory hierarchy beyond GPU HBM and CPU DRAM. Figure 3 illustrates the hierarchy emphasized by this literature.

*3.4.1 Processing in memory and near-data processing.* SmartSSD-based search, NDSEARCH, and Chameleon represent a strong trend toward reducing data movement for vector search and retrieval-heavy RAG deployments [7, 20, 27]. These systems move filtering, graph traversal, or search acceleration closer to storage or specialized hardware, which can improve scalability and sometimes energy efficiency. The trade-off is platform specificity: the systems are effective, but often require custom runtimes, hardware assumptions, or co-designed index layouts.

*3.4.2 Second-tier memory and disk-resident vector search.* DiskANN, LM-DiskANN, FreshDiskANN, and LSM-VEC show how disk-native or second-tier-memory-aware index structures can maintain large searchable corpora without requiring everything to remain in DRAM or GPU memory [10, 13, 16, 17]. Their main value is scalability at low memory cost, while their main risk is I/O sensitivity. For production RAG, this trade-off is increasingly acceptable because the knowledge store is often too large or too dynamic for fully in-memory operation.

### 3.5 Application-driven RAG systems

The final category focuses on applications whose domain constraints reshape system design. SEFRQO applies RAG to query optimization, where the retrieved objects and utility function differ from open-domain text generation [26]. AixelAsk targets table question answering and demonstrates how structured data and stepwise reasoning change retrieval patterns and prompt construction needs [1]. These cases suggest that future RAG platforms will need flexible abstractions that support workload-specific indexing, caching, and scheduling choices rather than a single universal architecture.

## 4 CROSS-CUTTING DESIGN DIMENSIONS

### 4.1 Latency versus quality

Approximate retrieval, aggressive caching, and lookahead scheduling can reduce latency, but each may risk lower recall or worse answer quality. Quality-aware systems such as METIS and AquaPipe are notable because they expose this trade-off directly rather than assuming that the fastest execution path is always acceptable [2, 19]. A mature RAG-serving stack should select operating points based on service-level objectives and quality targets.

### 4.2 Throughput versus memory capacity

Higher throughput often requires more memory for larger GPU-resident indexes, pipeline buffers, or caches. Yet keeping everything in fast memory is frequently impossible. This tension explains the rise of heterogeneous designs that combine GPUs, CPUs, CXL-like expansion, SSDs, and near-data accelerators [7, 10, 27]. Memory hierarchy management is therefore central rather than peripheral.

### 4.3 Static indexing versus dynamic updates

Production knowledge bases change continuously. Efficient support for online updates remains difficult because caches, vector indexes, and storage layouts all need to adapt together. SPFresh, EdgeRAG, FreshDiskANN, and LSM-VEC indicate that freshness support is becoming a central requirement for production-scale RAG systems [11, 13, 17, 28].

### 4.4 Monolithic versus heterogeneous architectures

Some systems target tightly integrated single-node performance, while others distribute work across CPUs, GPUs, storage devices, or edge-cloud deployments. Heterogeneous architectures can scale further and use memory more efficiently, but they increase scheduling complexity and widen the operating-design space [7, 9, 24]. The best architecture depends strongly on workload and deployment constraints.

### 4.5 Retrieval-centric versus generation-centric optimization

Many papers begin from either the ANN side or the LLM serving side, but the strongest recent systems increasingly optimize both together. RAGO, Hermes, Patchwork, and HyperRAG all emphasize that chunk selection, prompt assembly, cache reuse, and decode-time scheduling interact in ways that make stage-local optimization insufficient [14, 15, 21, 25].

## 5 COMPARATIVE VIEW OF REPRESENTATIVE SYSTEMS

Table 1 summarizes representative systems from the repository and highlights their main optimization targets and trade-offs.

The table highlights a broad trajectory in the literature. Earlier work often isolates a specific bottleneck such as GPU ANN search or disk-resident indexing, while newer systems combine ideas from caching, scheduling, and heterogeneous-memory management. This convergence suggests that future RAG platforms will increasingly resemble integrated data systems rather than loosely coupled model pipelines.

## 6 RESEARCH TRENDS

The repository literature suggests five major trends. First, RAG is becoming a systems problem rather than a retrieval-only problem [14, 21, 25]. Second, cache reuse is now a first-class design principle rather than a minor serving optimization [3, 4, 15, 18]. Third, memory hierarchy management is central because GPU memory alone is insufficient for many realistic deployments [7, 10, 27]. Fourth, near-data and storage-aware acceleration are becoming practical for large retrieval-heavy workloads [17, 20, 27]. Fifth, quality-aware adaptation is emerging as a necessary control layer for production systems that must balance speed and answer quality [2, 19].

## 7 OPEN CHALLENGES AND FUTURE DIRECTIONS

Despite rapid progress, several problems remain open.

- **Unified evaluation methodology.** Different papers use different datasets, retrieval settings, concurrency assumptions, and quality metrics, which makes direct comparison difficult.
- **Joint optimization of retrieval and decoding.** Many systems still optimize retrieval and generation separately, even though chunk selection, prompt packing, and decode scheduling interact strongly [15, 21, 25].
- **Dynamic and continuously updated knowledge bases.** Freshness support remains difficult once caches, indexes, and storage layouts must all evolve together [11, 17, 28].
- **Long-context and memory-intensive RAG.** Larger contexts amplify prompt assembly cost and KV-cache pressure, pushing future systems toward compression, reuse, and richer memory hierarchies [7, 18].
- **Multi-tenant production serving.** Tenant isolation, cost-aware scheduling, and service-level management remain underexplored despite their practical importance.
- **Application-specific architectures.** Query optimization, table reasoning, coding assistance, enterprise QA, and edge deployment may each require different retrieval and serving abstractions [1, 9, 26].

## 8 CONCLUSION

RAG-based LLM inference has evolved into a rich systems research area. Efficient deployment depends on much more than retrieval accuracy: GPU-native vector search, cache-centric optimization, pipeline scheduling, external-memory support, and application-aware design all shape the final latency, throughput, quality, and cost of a service. The literature collected in this repository makes one point especially clear: future RAG systems will be won by co-design across retrieval, generation, memory hierarchy, and orchestration. That integrated perspective is the central takeaway of this survey.

## REFERENCES

[1] AixelAsk 2026. AixelAsk: A Stepwise-Guided Retrieval and Reasoning Framework for Large Table QA. https://dl.acm.org/doi/10.1145/3769831 SIGMOD 2026.

[2] AquaPipe 2025. AquaPipe: A Quality-Aware Pipeline for Knowledge Retrieval and Large Language Models. https://dl.acm.org/doi/abs/10.1145/3709661 SIGMOD 2025.
[3] Cache-Craft 2025. Cache-Craft: Managing Chunk-Caches for Efficient Retrieval-Augmented Generation. https://arxiv.org/pdf/2502.15734v1 SIGMOD 2025.
[4] CacheBlend 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. https://arxiv.org/abs/2405.16444 EuroSys 2025.
[5] CacheFocus 2025. CacheFocus: Dynamic Cache Re-Positioning for Efficient Retrieval-Augmented Generation. https://arxiv.org/pdf/2502.11101 arXiv 2025.
[6] CAGRA 2024. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs. https://arxiv.org/pdf/2308.15136 ICDE 2024.
[7] Chameleon 2025. Chameleon: a Heterogeneous and Disaggregated Accelerator System for Retrieval-Augmented Language Models. https://www.vldb.org/pvldb/vol18/p42-jiang.pdf PVLDB 2025.
[8] CPU-GPU-Collab 2025. Towards High-throughput and Low-latency Billion-scale Vector Search via CPU/GPU Collaborative Filtering and Re-ranking. https://www.usenix.org/system/files/fast25-tian-bing.pdf FAST 2025.
[9] DGRAG 2025. DGRAG: Distributed Graph-based Retrieval-Augmented Generation in Edge-Cloud Systems. https://arxiv.org/abs/2505.19847 arXiv 2025.
[10] DiskANN 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. https://papers.nips.cc/paper_files/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html NeurIPS 2019.
[11] EdgeRAG 2024. EdgeRAG: Online-Indexed RAG for Edge Devices. https://arxiv.org/pdf/2412.21023 arXiv 2024.
[12] ELERAG 2025. ELERAG: Efficient Retrieval-Augmented Generation Inference with Lookahead Retrieval. https://arxiv.org/abs/2502.20969 arXiv 2025.
[13] FreshDiskANN 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. https://arxiv.org/abs/2105.09613 arXiv 2021.
[14] Hermes 2025. Hermes: Algorithm-System Co-design for Efficient Retrieval Augmented Generation At Scale. https://michaeltshen.github.io/Files/Hermes.pdf ISCA 2025.
[15] HyperRAG 2025. HyperRAG: Enhancing Quality-Efficiency Tradeoffs in Retrieval-Augmented Generation with Reranker KV-Cache Reuse. https://arxiv.org/pdf/2504.02921 arXiv 2025.
[16] LM-DiskANN 2023. LM-DiskANN: Low Memory Footprint in Disk-Native Dynamic Graph-Based ANN Indexing. https://ieeexplore.ieee.org/document/10386517 BigData 2023.
[17] LSM-VEC 2025. LSM-VEC: A Large-Scale Disk-Based System for Dynamic Vector Search. https://arxiv.org/pdf/2505.17152 SOSP 2025.
[18] MemoRAG 2025. MemoRAG: Boosting Long Context Processing with Global Memory-Enhanced Retrieval Augmentation. https://dl.acm.org/doi/abs/10.1145/3696410.3714805 WWW 2025.
[19] METIS 2025. METIS: Fast Quality-Aware RAG Systems with Configuration Adaptation. https://arxiv.org/pdf/2412.10543 SOSP 2025.
[20] NDSEARCH 2024. NDSEARCH: Accelerating Graph-Traversal-Based Approximate Nearest Neighbor Search through Near Data Processing. https://arxiv.org/pdf/2312.03141 ISCA 2024.
[21] Patchwork 2025. Patchwork: A Unified Framework for RAG Serving. https://arxiv.org/pdf/2505.07833 arXiv 2025.
[22] PilotANN 2025. PilotANN: Memory-Bounded GPU Acceleration for Vector Search. https://arxiv.org/pdf/2503.21206 arXiv 2025.
[23] PipeRAG 2025. PipeRAG: Fast Retrieval-Augmented Generation via Adaptive Pipeline Parallelism. https://arxiv.org/pdf/2403.05676 KDD 2025.
[24] RAGDoll 2025. RAGDoll: Efficient Offloading-based Online RAG System on a Single GPU. https://arxiv.org/abs/2504.15302 arXiv 2025.
[25] RAGO 2025. RAGO: Systematic Performance Optimization for Retrieval-Augmented Generation Serving. https://arxiv.org/abs/2503.14649 ISCA 2025.
[26] SEFRQO 2026. SEFRQO: A Self-Evolving Fine-Tuned RAG-Based Query Optimizer. https://dl.acm.org/doi/10.1145/3769826 SIGMOD 2026.
[27] SmartSSD 2024. Scalable Billion-point Approximate Nearest Neighbor Search Using SmartSSDs. https://www.usenix.org/system/files/atc24-tian.pdf USENIX ATC 2024.
[28] SPFresh 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. https://dl.acm.org/doi/abs/10.1145/3600006.3613166 SOSP 2023.

[29] VecFlow 2026. VecFlow: A High-Performance Vector Data Management System for Filtered-Search on GPUs. https://arxiv.org/pdf/2506.00812 SIGMOD 2026.

Table 1. Representative systems and their primary design trade-offs.

| Paper | Year | Main target | Optimization level | Hardware setting | Main benefit | Key trade-off |
|---|---|---|---|---|---|---|
| RAGO [25] | 2025 | End-to-end RAG serving | System pipeline | GPU | Lower latency and higher throughput | Additional orchestration complexity |
| Hermes [14] | 2025 | Efficient large-scale RAG | Algorithm–system co-design | GPU | Better accelerator utilization | GPU-memory dependence |
| CAGRA [6] | 2024 | GPU ANN search | Index and kernel design | GPU | High parallel vector search | Limited by accelerator capacity |
| CacheCraft [3] | 2025 | Chunk-cache management | Cache reuse | CPU/GPU serving | Fewer repeated retrievals | Freshness and cache sizing |
| CacheBlend [4] | 2025 | Cached knowledge fusion | Model-serving reuse | GPU/LLM serving | Faster serving | Cache validity |
| MemoRAG [18] | 2025 | Long-context reuse | Memory-enhanced retrieval | Heterogeneous serving | Better context reuse | Added state-management overhead |
| METIS [19] | 2025 | Quality-aware RAG | Adaptive control | Heterogeneous | Better quality-efficiency control | Controller complexity |
| PipeRAG [23] | 2025 | Adaptive pipelining | Scheduling | Heterogeneous | Retrieval-generation overlap | Backpressure and balancing |
| Patchwork [21] | 2025 | Unified RAG serving | Framework design | Heterogeneous | Integrated end-to-end serving | Wider orchestration scope |
| SmartSSD search [27] | 2024 | Billion-scale retrieval | Near-data processing | SmartSSD | Scalability beyond host memory | Hardware specialization |
| Chameleon [7] | 2025 | Retrieval-augmented LMs | Disaggregated acceleration | Heterogeneous accelerators | Better memory scalability | Deployment complexity |
| DiskANN [10] | 2019 | Disk-native ANN | Index + I/O design | SSD | Large-scale low-memory search | I/O-sensitive performance |
| LSM-VEC [17] | 2025 | Dynamic vector search | Storage-aware updates | SSD + memory | Better update support at scale | More complex maintenance path |
| SEFRQO [26] | 2026 | Query optimization | Application-specific RAG | Database setting | Domain-aware retrieval benefits | Narrower workload scope |